

### 3 – CONTROL STRUCTURES: SEQUENTIAL AND CONDITIONALS

#### Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** control structures and explain their role in determining the flow of execution in a program.
- **Differentiate** between the four main categories of control structures: sequential, conditional/branching, repetition/looping, and function/method/procedure.
- **Describe** how sequential execution works and explain why it is the foundation of all programs.
- **Construct** `if`, `elif`, and `else` statements to implement two-way and multi-way branching.
- **Apply** comparison operators (`>`, `<`, `>=`, `<=`, `==`, `!=`) correctly to values of different types, including numbers, strings, booleans, and lists.
- **Combine** conditions using the logical operators `and`, `or`, and `not`, and explain short-circuit evaluation.
- **Write** ternary expressions for simple conditional assignments and recognize when they reduce readability.
- **Explain** the purpose of bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`) and identify scenarios where they are appropriate.
- **Use** the `match` statement (Python 3.10+) for multi-way branching, including literal patterns, OR patterns, guards, and structural patterns.
- **Identify** and describe keywords that alter control flow (`break`, `continue`, `return`) and their effect within control structures.
- **Recognize** the role of indentation in Python and the risks associated with inconsistent whitespace when pasting code.
- **Evaluate** when to use `match` versus `if-elif` chains based on readability and structural complexity.

#### Introduction

Control structures are fundamental programming constructs that determine how and in what order your code executes. They allow you to control the flow of your program rather than having it simply run from top to bottom in a straight line.

Understanding control structures allows you to

- **Predict** program behavior
- **Write** flexible programs

- **Build** logic that responds to different inputs and situations

The main control structures:

1. **Sequence/Sequential:**  
The statements are processed in order with no skips or jumps.
2. **Conditional/Branching:**  
Certain blocks of code are executed **only if** a condition is true
3. **Repetition/Looping/Iteration:**  
Statements or blocks of statements that are executed zero or more times
4. **Function/Method/Procedure:**  
A named block of code that runs only when called

Control structures may be **nested** i.e. you may put a loop in an `if` statement or put a `switch` statement inside of a loop to achieve complex behaviors. However, in this course we will not define a function inside any other controls structure including itself.

Keywords that Alter Control Flow

The following keywords alter the normal flow inside of control structures:

- **break** – exits a loop early.
- **continue** – skips to the next iteration of a loop.
- **return** – exits a function and optionally returns a value

Each control structure adds expressive power to the language.

### 3.a – Sequential

Sequential execution is the simplest and is the default behavior of a program. Statements execute one after another, exactly one, in order they appear.

This guarantees that all the statements will be processed exactly once. It serves as the foundation of most processing techniques.

Because all the statements are processed in the same order this control structure will do the exact thing every time it is executed (i.e. behave the same way every time it is processed). So additional control structure is needed to build any substantial program i.e., one that can do at least two or more tasks or to behave differently every time you run it.

```
#set of statements without skips or repeats
>>> name = input('Please enter your name: ')
Please enter your name: Narendra
```

```
>>> program = input('Please enter your program: ')
Please enter your program: Game - Programming

>>> print(f'Hello {name} welcome to {program}')
Hello Narendra welcome to Game - Programming
```

The above code will always do the same thing: prompt for a string, read the string prompt for a second string and read that also. Then it prints a message. Programs written with only sequence control structure is capable of completing only a single task.

### 3.b – Conditionals

This control structure allows you to process or ignore code blocks depending (true/false blocks). This gives immense power to the language, now your programs are able to complete multiple/different tasks. Simply implement the individual tasks in different blocks. If there are more tasks to be implement then nest more conditionals.

Like most other modern languages there is a match statement that does the same job as a switch statement in other languages.

Notice unlike C# there is no parenthesis around the assertion nor are there curly braces to delimit the true or the else code blocks. Blocks are defined by indentations. Most IDE's will manage indentations for you (VS Code use 4 spaces for an indent), but if you paste code from another source, then there could be problems. Visually you may not be able to discern between 4 spaces and a tab, the Python interpreter will complain most vehemently.

#### 3.b.1 – if

This is the basic syntax to test if a condition is true. If so, the indented code will be executed, if not it will be skipped.

```
n = 5
if n > 2:
    print('Larger than 2')
```

### 3.b.2 – else

This is a way to specify some code that will be executed if a condition is false.

```
age = 18
if age > 16:
    print('Can drive')
else:
    print("Don't drive")
```

### 3.b.3 – elif

In addition to the initial If statement condition, you can add extra conditions to test if the first condition is false.

Once an elif condition is true, the rest of the elif conditions are no longer checked and are skipped.

```
weather = 'sunny'
if weather == 'rain':
    print('bring umbrella')
elif weather == 'sunny':
    print('bring sunglasses')
elif weather == 'snow':
    print('bring gloves')
else:
    print('bring everything')
```

### 3.b.4 – and

This expects the conditions on both sides of the and to be true.

```
s = 58
if s < 60 and s > 50:
    print('Your grade is C')
```

### 3.b.5 – or

This expects either of the conditions on both sides of the or to be true. Basically, both conditions cannot be false.

```
age = 12
if age < 16 or age > 200:
    print("Can't drive")
```

### 3.b.6 – not

This will flip the original result of the condition. e.g. if it was true then it's now false.

```
if not 3 > 1:
    print('something')
#Will not be printed.
```

### 3.b.7 – Ternary statements

The *ternary statement* is a short hand if-else statement.

```
#ternary statement
«value if assertion is true» «assertion» «value if assertion is false»:

status = 'Wealthy' if asset > 1_000_000 else 'Normal'
```

There is a short hand version

```
#ternary statement
>>> a
```

```
345
>>> a or 'Hello'
345

>>> a and 'Hello'
'Hello'
```

Best practice:

- **Use only for simple choices**
- Avoid nesting more than once — prefer `if` block when readability suffers
- Never use the confusing `x or y / x and y` trick as a substitute — it is error-prone

### 3.c – Comparison Operators

These comparison (relational) operators allow you to compare two similarly typed values and either return `True` or `False`. They work the same way as in other languages. Python compares string **lexicographically** and list **element-by-element** (first difference decides).

#### 3.c.1 – Greater than (>) operator

This operator will return `True` if the left side is greater than the right side otherwise it will return `False`.

```
>>> -2 > 3
False

>>> 1 > 1
False

>>> 3 > 2
True

>>> 1.2 > 1.4
```

```
False

>>> 3.1 > 3.1
False

>>> 9.8 > 7.2
True

>>> False > True
False

>>> True > True
False

>>> True > False
True

>>> 'apple' > 'zebra'
False

>>> 'zebra' > 'apple'
True

>>> 'baby' > 'baby'
False

>>> [4, 9] > [5, 0]
False

>>> [2, 3] > [2, 3]
False

>>> [5] > [4]
True
```

### 3.c.2 – Less than (<) operator

This operator will return True if the left side is less than the right side otherwise it will return False.

```
>>> -2 < 3
True

>>> 1 < 1
False

>>> 1.2 < 1.4
True

>>> False < True
True

>>> 'apple' < 'zebra'
True

>>> 'baby' < 'baby'
False

>>> [4, 9] < [5, 0]
True
```

### 3.c.3 – Greater than or equal (>=) operator

This operator will return True if the left side is greater than or equal to the right side otherwise it will return False.

```
>>> 3 >= 2
True

>>> 1.2 >= 1.4
False
```

```
>>> 3.1 >= 3.1
True
```

```
>>> 9.8 >= 7.2
True
```

### 3.c.4 – Less than or equal (<=) operator

This operator will return True if the left side is less than or equal right side otherwise it will return False.

```
>>> -2 <= 3
True
```

```
>>> 1.2 <= 1.4
True
```

```
>>> False <= True
True
```

```
>>> 'apple' <= 'zebra'
True
```

```
>>> [4, 9] <= [5, 0]
True
```

### 3.c.5 – Equality (==) operator

This operator will return True only if the left side is the same as the right side otherwise it will return False.

```
>>> 1 == 1
True
```

```
>>> 3.1 == 3.1
True
```

```
>>> 9.8 == 7.2
False

>>> False == False
True

>>> True == True
True

>>> 'baby' == 'baby'
True

>>> [2, 3] == [2, 3]
True
```

### 3.c.6 – Not equals (!=) operator

Return True if the left and right side values are not equal.

```
>>> -2 != 3
True

>>> 1 != 1
False

>>> 9.8 != 7.2
True

>>> True != False
True

>>> 'zebra' != 'apple'
True

>>> 'baby' != 'baby'
```

```
False
```

```
>>> [5] != [4]
```

```
True
```

### 3.d – Logic Operators

These operators allow you to implement more complex logic. These operators either the first argument or the second argument. The “and” and the “or” operators take two arguments while the “not” operator takes a single expression.

#### Fun fact

These operators behave very differently from most other languages.

#### 3.d.1 – Logical ‘and’ operator

The expression “x and y” first evaluates \*x\*; if \*x\* is false, its value is returned; otherwise, \*y\* is evaluated and the resulting value is returned.

```
# and returns true if both expressions are true, false otherwise
```

```
>>> age = 20
```

```
>>> province = "QC"
```

```
>>> age >= 18 and province == "ON"
```

```
False
```

#### 3.d.2 – Logical ‘or’ operator

The expression “x or y” first evaluates \*x\*; if \*x\* is true, its value is returned; otherwise, \*y\* is evaluated and the resulting value is returned.

```
# or returns true if either expression is true, false otherwise
>>> age = 50
>>> service = 6

>>> age > 65 or service > 10
False
```

### 3.d.3 – Logical 'not' operator

The operator "not" yields "True" if its argument is false, "False" otherwise. Thus, this operator takes a single argument and return either True or False.

```
# not flips the bool value

>>> age = 50

>>> not age > 65
True
```

### 3.d.3.1 – Bit fiddling (This sub-section covers an advanced topic. You can safely ignore it.)

Bit fiddling (also known as bit manipulation or bit twiddling) in Python refers to the use of **bitwise operators** to directly manipulate the individual binary digits (bits) of an integer. This technique allows for efficient storage and manipulation of data at a low level, which is useful in specific scenarios like networking, cryptography, and embedded systems programming.

#### 3.d.3.1.1 – Bitwise Operators

The bitwise operators are (&, |, ^, ~, << and >>) These operators take two integers and returns an integer. It treats numbers as a sequence of binary digits (bits) and perform operations bit by bit.

To get a better understanding binary values are used in the demonstration.

Please be advised these operators will only work with operands of the int type.

### 3.d.3.1.2 – Bitwise AND (&) Operator

The bitwise AND operator compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 & 3      # and operator
1              # 1001 & 0011 = 0001

>>> 15 & 3
3              # 1111 & 0011 = 0011
```

### 3.d.3.1.3 – Bitwise OR (|) Operator

The bitwise OR operator compares each bit of the first operand to the corresponding bit of the second operand. If either bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 | 3      # or operator
11             # 1001 | 0011 = 1011

>>> 15 | 3
15             # 1111 | 0011 = 1111
```

### 3.d.3.1.4 – Bitwise XOR (^) Operator

The bitwise XOR operator compares each bit of the first operand to the corresponding bit of the second operand. If the bits are not the same, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 ^ 3      # xor operator
10             # 1001 ^ 0011 = 1010

>>> 15 ^ 3
12             # 1111 ^ 0011 = 1100
```

#### 3.d.3.1.5 – Bitwise NOT (~) Operator

The bitwise NOT operator is a unary operator that flips the bits of its operand . Python uses **two's complement**, ( $\sim x = -x - 1$ ) not simple bit flipping with fixed width).

```
>>> ~3      # not operator
-4          # result is based on two's complement representation
           # ~0b11 = 100
```

#### 3.d.3.1.6 – Left Shift (<<) Operator

The left shift operator shifts the bits of the number to the left and fills 0 on voids left as a result.

```
>>> 9 << 1   # left shift operator
18          # 1001 <= 2 = 10010 (binary)

>>> 9 << 2
36          # 1001 <= 4 = 100100 (binary)
```

#### 3.d.3.1.7 – Right Shift (>>) Operator

The right shift operator shifts the bits of the number to the right and fills 0 on voids left as a result.

```
>>> 9 >> 1    # right shift operator
4            # 1001 => 2 = 100 (binary)

>>> 9 >> 2
2            # 1001 => 4 = 10 (binary)
```

### 3.d.4 – Best Practice

- Use bitwise operators sparingly and only when necessary
- Avoid using bitwise operators to perform arithmetic operations, as this can make your code harder to understand.

### 3.e – Match statement

The `if` statement allows only two-way branching. In the situation where multi-way branching is required, then the `match` statement can be used.

This was introduced in Python 3.10 and is similar in purpose and working as the `switch` statement is most other languages.

```
day = input('Enter the three letter day of the week: ')

match(day.lower()):
    #make it lowercase
    case 'sat' | 'sun':
        #two cases combined using
        print(f'{day} is a family day')
    case _:
        #this will catch the remaining cases
        print(f'{day} is a work day')
```

```
x = int(input('Enter a number between 1 and 20: '))

match x:
    case 1:
        print('one')
    case 2 | 3:
        print('two or three')
    case x if x % 5 == 0:
        print('multiple of 5')
    case x if x > 16:
        print('greater than 16')
    case _:
        print('Value not in range')
```

```
name = input('Enter all your names: ')
names = name.split()
match names:
    case [x, y]:
        print(f'two names {x} {y}')
    case [x, y, z]:
        print(f'three names {x} {y} {z}')
    case [x, *y, z]:
        print(f"more than three names {x} {' '.join(y)} {z}")
    case _:
        print('Unkown error')
```

`match` is often more readable than long `if-elif` chains, especially when checking structure statement is more readable than the corresponding `if`-statement and in some (advanced) cases it is more powerful.

### 3.f – Summary

This chapter introduced **control structures**, the constructs that determine the order in which a Python program executes. Without them, a program can only run top-to-bottom and perform a single task; with them, programs can make decisions, repeat work, and respond to different inputs.

Key takeaways:

- **Sequential execution** is the default: statements run once, in order, with no skips. It is the foundation of all programs but is limited to a single, fixed behavior.
- **Conditionals** (`if`, `elif`, `else`) allow branching based on true/false conditions. Blocks are defined by indentation, not braces. The `elif` chain stops at the first true condition.
- **Logical operators** (`and`, `or`, `not`) build compound conditions. `and` and `or` use short-circuit evaluation and return one of their operands; `not` always returns a bool.
- **Comparison operators** (`>`, `<`, `>=`, `<=`, `==`, `!=`) compare values and return `True` or `False`. They work on numbers, strings (lexicographically), booleans, and lists (element-by-element).

- **Ternary expressions** (`value_if_true if condition else value_if_false`) provide compact one-line branching, but should be used sparingly for readability.
- **Bitwise operators** (`&`, `|`, `^`, `~`, `<<`, `>>`) manipulate integers at the binary level. They are powerful but specialized — best reserved for low-level tasks like networking, cryptography, and embedded systems.
- **The match statement** (Python 3.10+) enables clean multi-way branching, similar to `switch` in other languages. It supports literal patterns, OR patterns (`|`), guards (`case x if ...`), and structural patterns for sequences.
- **Loop-altering keywords** (`break`, `continue`, `return`) modify normal flow inside control structures.

### 3.g – Non coding exercises

1. Which control structure guarantees each statement runs exactly once, in order?
2. What does `elif` do?
3. Which operator returns `True` only when both sides are true?
4. What does `~3` evaluate to in Python?
5. Which Python version introduced the `match` statement?
6. What does the `break` keyword do?
7. In Python, code blocks are delimited by curly braces `{ }`. (True or false?)
8. `and` and `or` always return a boolean value. (True or false?)
9. `1.2 <= 1.4` evaluates to `True`. (True or false?)
10. Bitwise operators work on floats as well as ints. (True or false?)
11. `match` can replace long `if-elif` chains and is often more readable. (True or false?)
12. `not` is a binary operator. (True or false?)
13. What is the result of `5 > 3 and 2 > 4`?
14. What is the result of `not (3 == 3)`?
15. What is the result of `'apple' < 'banana'`?
16. What is the result of `[1, 2] < [1, 3]`?
17. What is the result of `9 & 5` (in binary: `1001 & 0101`)?
18. What is the result of `9 | 5`?
19. What is the result of `9 ^ 5`?
20. What is the result of `4 << 2`?
21. What is the result of `20 >> 2`?
22. Why sequential-only programs are limited?
23. Why does Python rely on indentation instead of braces? What risk does this create when pasting code?
24. Give an example of a real-world decision that maps naturally to an `if-elif-else` chain.
25. When would you choose `match` over `if-elif`? Give one scenario.

- 26. Why should bitwise operators be used sparingly?
- 27. What is short-circuit evaluation, and why is it useful?